Numerical Linear Algebra II: 08 PageRank Algorithm

Jakub Kruzik
jakub.kruzik@vsb.cz

2026
Ostrava

Introduction: Network Centrality Beyond the Web

Introduced by Page and Brin (1998) for Google Search, **PageRank** measures the global importance (centrality) of nodes in any directed graph $G = (V, E)$ from its link structure:

Introduction: Network Centrality Beyond the Web

Introduced by Page and Brin (1998) for Google Search, **PageRank** measures the global importance (centrality) of nodes in any directed graph $G = (V, E)$ from its link structure:

- **World Wide Web:** Ranking web pages by hyperlink authority.

Introduction: Network Centrality Beyond the Web

Introduced by Page and Brin (1998) for Google Search, **PageRank** measures the global importance (centrality) of nodes in any directed graph $G = (V, E)$ from its link structure:

- **World Wide Web:** Ranking web pages by hyperlink authority.
- **Bibliometrics & Citations:** Impact of papers, authors, or journals.

Introduction: Network Centrality Beyond the Web

Introduced by Page and Brin (1998) for Google Search, **PageRank** measures the global importance (centrality) of nodes in any directed graph $G = (V, E)$ from its link structure:

- **World Wide Web:** Ranking web pages by hyperlink authority.
- **Bibliometrics & Citations:** Impact of papers, authors, or journals.
- **Social & Trust Networks:** Identifying influential users and trust propagation.

Introduction: Network Centrality Beyond the Web

Introduced by Page and Brin (1998) for Google Search, **PageRank** measures the global importance (centrality) of nodes in any directed graph $G = (V, E)$ from its link structure:

- **World Wide Web:** Ranking web pages by hyperlink authority.
- **Bibliometrics & Citations:** Impact of papers, authors, or journals.
- **Social & Trust Networks:** Identifying influential users and trust propagation.
- **Bioinformatics:** Prioritizing genes or proteins in PPI networks (*GeneRank*).

Introduction: Network Centrality Beyond the Web

Introduced by Page and Brin (1998) for Google Search, **PageRank** measures the global importance (centrality) of nodes in any directed graph $G = (V, E)$ from its link structure:

- **World Wide Web:** Ranking web pages by hyperlink authority.
- **Bibliometrics & Citations:** Impact of papers, authors, or journals.
- **Social & Trust Networks:** Identifying influential users and trust propagation.
- **Bioinformatics:** Prioritizing genes or proteins in PPI networks (*GeneRank*).
- **Recommender Systems & Infrastructure:** Personalized rankings of user items; critical nodes in road, power-grid, and financial networks.

Directed Graph and Adjacency Matrix $\boldsymbol{A}$

We represent $G = (V, E)$ ($N = |V|$) by its binary **adjacency matrix** $\boldsymbol{A} \in \{0, 1\}^{N \times N}$:

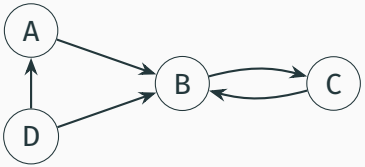
$$A_{ij} = \begin{cases} 1 & \text{if node } j \text{ links to node } i \ (j \to i), \\ 0 & \text{otherwise.} \end{cases}$$

Directed Graph and Adjacency Matrix $\boldsymbol{A}$

We represent $G = (V, E)$ ($N = |V|$) by its binary **adjacency matrix** $\boldsymbol{A} \in \{0,1\}^{N \times N}$:

$$A_{ij} = \begin{cases} 1 & \text{if node } j \text{ links to node } i \ (j \to i), \\ 0 & \text{otherwise.} \end{cases}$$

Column j lists outgoing links from j ($\text{out}(j) = \sum_{i=1}^{N} A_{ij}$); **row** i lists incoming links to i:



$$\boldsymbol{A} = \begin{array}{c} \\ A \\ B \\ C \\ D \end{array} \begin{array}{c} \begin{array}{cccc} A & B & C & D \end{array} \\ \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \end{array}$$

PageRank Score $PR(x)$ and Core Principles

PageRank assigns to every node $x \in \{1, \ldots, N\}$ a non-negative **importance score** $PR(x) \geq 0$ normalized so that $\sum_{x=1}^{N} PR(x) = 1$:

PageRank Score $PR(x)$ and Core Principles

PageRank assigns to every node $x \in \{1, \ldots, N\}$ a non-negative **importance score** $PR(x) \geq 0$ normalized so that $\sum_{x=1}^{N} PR(x) = 1$:

1. **Link Democracy (Links as Votes):** A directed edge $y \to x$ is a vote of confidence from node y to node x.

PageRank Score $PR(x)$ and Core Principles

PageRank assigns to every node $x \in \{1, \ldots, N\}$ a non-negative **importance score** $PR(x) \geq 0$ normalized so that $\sum_{x=1}^{N} PR(x) = 1$:

1. **Link Democracy (Links as Votes):** A directed edge $y \to x$ is a vote of confidence from node y to node x.
2. **Recursive Importance:** A vote from an important node carries more weight, and node y splits its score $PR(y)$ equally among its $\text{out}(y) > 0$ outgoing links:

$$PR(x) = \sum_{y \to x} \frac{PR(y)}{\text{out}(y)}, \qquad x = 1, \ldots, N.$$

The Column-Stochastic Transition Matrix $\boldsymbol{M}$

Assuming $\text{out}(j) \geq 1$ for all j, normalizing each column j of $\boldsymbol{A}$ by $\text{out}(j)$ gives the **transition (Markov) matrix** $\boldsymbol{M} \in \mathbb{R}^{N \times N}$:

$$M_{ij} = \frac{A_{ij}}{\text{out}(j)} = \begin{cases} \dfrac{1}{\text{out}(j)} & \text{if } j \to i, \\ 0 & \text{otherwise,} \end{cases} \qquad \iff \qquad \boldsymbol{M} = \boldsymbol{A}\boldsymbol{D}_{\text{out}}^{-1},$$

The Column-Stochastic Transition Matrix $\boldsymbol{M}$

Assuming $\text{out}(j) \geq 1$ for all j, normalizing each column j of $\boldsymbol{A}$ by $\text{out}(j)$ gives the **transition (Markov) matrix** $\boldsymbol{M} \in \mathbb{R}^{N \times N}$:

$$M_{ij} = \frac{A_{ij}}{\text{out}(j)} = \begin{cases} \dfrac{1}{\text{out}(j)} & \text{if } j \to i, \\ 0 & \text{otherwise,} \end{cases} \qquad \Longleftrightarrow \qquad \boldsymbol{M} = \boldsymbol{A}\boldsymbol{D}_{\text{out}}^{-1},$$

where $\boldsymbol{D}_{\text{out}} = \text{diag}(\text{out}(1), \ldots, \text{out}(N))$.

- $M_{ij} \geq 0$ and every column sums to 1 ($\boldsymbol{e}^T\boldsymbol{M} = \boldsymbol{e}^T$), so $\boldsymbol{M}$ is **column-stochastic**.
- Consequently, $\rho(\boldsymbol{M}) = 1$ and $\mu_1 = 1$ is the dominant eigenvalue of $\boldsymbol{M}$.

Random Walk $\boldsymbol{p}(t)$, Stationary Vector $\boldsymbol{r}$, and $PR(x)$

Consider a **random walker** (surfer) following outgoing links uniformly at random:

1. **State distribution at step t:** Let $p_x(t) = \mathbb{P}(\text{walker at node } x \text{ at step } t)$, stored in $\boldsymbol{p}(t) = (p_1(t), \ldots, p_N(t))^T$ with $\boldsymbol{e}^T\boldsymbol{p}(t) = 1$. One step of the walk satisfies:

$$p_x(t+1) = \sum_{y \to x} \frac{p_y(t)}{\text{out}(y)} = \sum_{y=1}^{N} M_{xy} p_y(t) \quad \Longleftrightarrow \quad \boxed{\boldsymbol{p}(t+1) = \boldsymbol{M}\boldsymbol{p}(t).}$$

Random Walk $\boldsymbol{p}(t)$, Stationary Vector $\boldsymbol{r}$, and $PR(x)$

Consider a **random walker** (surfer) following outgoing links uniformly at random:

1. **State distribution at step t:** Let $p_x(t) = \mathbb{P}(\text{walker at node } x \text{ at step } t)$, stored in $\boldsymbol{p}(t) = (p_1(t), \ldots, p_N(t))^T$ with $\boldsymbol{e}^T\boldsymbol{p}(t) = 1$. One step of the walk satisfies:

$$p_x(t+1) = \sum_{y \to x} \frac{p_y(t)}{\text{out}(y)} = \sum_{y=1}^{N} M_{xy} p_y(t) \quad \Longleftrightarrow \quad \boxed{\boldsymbol{p}(t+1) = \boldsymbol{M}\boldsymbol{p}(t).}$$

2. **Stationary PageRank vector $\boldsymbol{r}$:** As $t \to \infty$, $\boldsymbol{p}(t)$ converges to the steady-state vector $\boldsymbol{r}$, whose x-th entry r_x is the PageRank score $PR(x)$:

$$\boxed{\boldsymbol{r} = \lim_{t\to\infty} \boldsymbol{p}(t) = (PR(1), \ldots, PR(N))^T, \qquad r_x = PR(x), \quad \boldsymbol{e}^T\boldsymbol{r} = 1.}$$

PageRank as the Dominant Eigenvector Problem

At steady state $\boldsymbol{p}(t+1) = \boldsymbol{p}(t) = \boldsymbol{r}$, the node balance equations $PR(x) = \sum_{y \to x} \frac{PR(y)}{\text{out}(y)}$ become the matrix eigenvalue equation:

$$r_x = \sum_{y=1}^{N} M_{xy} r_y \quad (\forall x) \qquad \Longleftrightarrow \qquad \boxed{\boldsymbol{M}\boldsymbol{r} = 1 \cdot \boldsymbol{r}, \qquad \|\boldsymbol{r}\|_1 = \boldsymbol{e}^T \boldsymbol{r} = 1.}$$

PageRank as the Dominant Eigenvector Problem

At steady state $\boldsymbol{p}(t+1) = \boldsymbol{p}(t) = \boldsymbol{r}$, the node balance equations $PR(x) = \sum_{y \to x} \frac{PR(y)}{\text{out}(y)}$ become the matrix eigenvalue equation:

$$r_x = \sum_{y=1}^{N} M_{xy} r_y \quad (\forall x) \qquad \Longleftrightarrow \qquad \boxed{\boldsymbol{M}\boldsymbol{r} = 1 \cdot \boldsymbol{r}, \qquad \|\boldsymbol{r}\|_1 = \boldsymbol{e}^T \boldsymbol{r} = 1.}$$

- **Spectral Interpretation:** The PageRank vector $\boldsymbol{r} = (PR(1), \ldots, PR(N))^T$ is the **dominant right eigenvector** of the Markov transition matrix $\boldsymbol{M}$ corresponding to eigenvalue $\mu_1 = 1$.

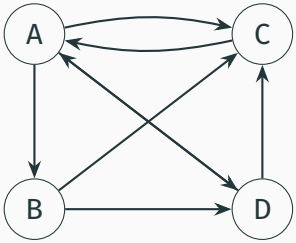
PageRank as the Dominant Eigenvector Problem

At steady state $\boldsymbol{p}(t+1) = \boldsymbol{p}(t) = \boldsymbol{r}$, the node balance equations $PR(x) = \sum_{y \to x} \frac{PR(y)}{\text{out}(y)}$ become the matrix eigenvalue equation:

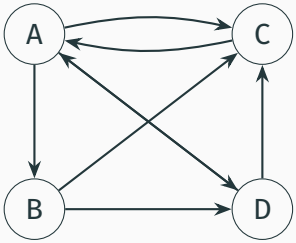
$$r_x = \sum_{y=1}^{N} M_{xy} r_y \quad (\forall x) \qquad \Longleftrightarrow \qquad \boxed{\boldsymbol{M}\boldsymbol{r} = 1 \cdot \boldsymbol{r}, \qquad \|\boldsymbol{r}\|_1 = \boldsymbol{e}^T \boldsymbol{r} = 1.}$$

- **Spectral Interpretation:** The PageRank vector $\boldsymbol{r} = (PR(1), \ldots, PR(N))^T$ is the **dominant right eigenvector** of the Markov transition matrix $\boldsymbol{M}$ corresponding to eigenvalue $\mu_1 = 1$.
- **Iterative Notation:** In iterative algorithms, the k-th power-method iterate $\boldsymbol{p}(k)$ has entries $PR_k(x) = p_x(k)$.

Example: 4-Node Graph and Balance Equations



Example: 4-Node Graph and Balance Equations



Out-degrees: $\text{out}(A) = 3$, $\text{out}(B) = 2$, $\text{out}(C) = 1$, $\text{out}(D) = 2$.

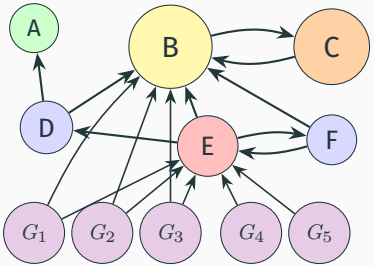
$$PR(A) = PR(C) + \tfrac{1}{2}PR(D)$$
$$PR(B) = \tfrac{1}{3}PR(A)$$
$$PR(C) = \tfrac{1}{3}PR(A) + \tfrac{1}{2}PR(B) + \tfrac{1}{2}PR(D)$$
$$PR(D) = \tfrac{1}{3}PR(A) + \tfrac{1}{2}PR(B)$$

Example: Dominant Eigenvector of the 4-Node Graph

In matrix form $\boldsymbol{M}\boldsymbol{r} = 1 \cdot \boldsymbol{r}$ with normalization $\boldsymbol{e}^T\boldsymbol{r} = 1$:

$$\underbrace{\begin{pmatrix} 0 & 0 & 1 & 1/2 \\ 1/3 & 0 & 0 & 0 \\ 1/3 & 1/2 & 0 & 1/2 \\ 1/3 & 1/2 & 0 & 0 \end{pmatrix}}_{\boldsymbol{M}} \underbrace{\begin{pmatrix} PR(A) \\ PR(B) \\ PR(C) \\ PR(D) \end{pmatrix}}_{\boldsymbol{r}} = \underbrace{\begin{pmatrix} PR(A) \\ PR(B) \\ PR(C) \\ PR(D) \end{pmatrix}}_{\boldsymbol{r}} \implies \boldsymbol{r} = \frac{1}{31}\begin{pmatrix} 12 \\ 4 \\ 9 \\ 6 \end{pmatrix} \approx \begin{pmatrix} 0.387 \\ 0.129 \\ 0.290 \\ 0.194 \end{pmatrix}.$$

Pathologies of Pure Link-Following ($\boldsymbol{p}(t+1) = \boldsymbol{M}\boldsymbol{p}(t)$)

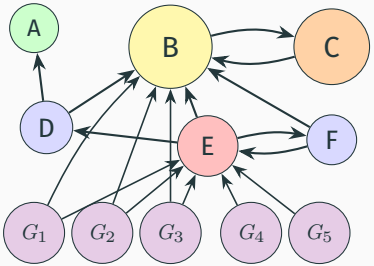


Random walk starting at G:

$$G \to E \to F \to E \to D \to B \leftrightarrow C$$

1. **Spider traps (cycles):** Closed cycle $B \leftrightarrow C$ traps the walker forever.

Pathologies of Pure Link-Following ($\boldsymbol{p}(t+1) = \boldsymbol{M}\boldsymbol{p}(t)$)



Random walk starting at G:

$$G \to E \to F \to E \to D \to B \leftrightarrow C$$

1. **Spider traps (cycles):** Closed cycle $B \leftrightarrow C$ traps the walker forever.
2. **Sink / dangling nodes:** Node A has $\text{out}(A) = 0$ (a zero column in $\boldsymbol{M}$, leaking probability mass).

Teleportation and the Damping Factor λ

To resolve spider traps and disconnected components, Brin and Page introduced a **damping factor** $\lambda \in (0, 1)$ (typically $\lambda = 0.85$):

Teleportation and the Damping Factor λ

To resolve spider traps and disconnected components, Brin and Page introduced a **damping factor** $\lambda \in (0, 1)$ (typically $\lambda = 0.85$):

- With probability λ, the walker follows an outgoing link at random.
- With probability $1 - \lambda$, the walker **teleports** (jumps) uniformly at random to any of the N nodes in the graph.

Teleportation and the Damping Factor λ

To resolve spider traps and disconnected components, Brin and Page introduced a **damping factor** $\lambda \in (0, 1)$ (typically $\lambda = 0.85$):

- With probability λ, the walker follows an outgoing link at random.
- With probability $1 - \lambda$, the walker **teleports** (jumps) uniformly at random to any of the N nodes in the graph.

Damped PageRank Equation

$$p_x(t+1) = \frac{1-\lambda}{N} + \lambda \sum_{y \to x} \frac{p_y(t)}{\text{out}(y)} \quad \xrightarrow{t \to \infty} \quad PR(x) = \frac{1-\lambda}{N} + \lambda \sum_{y \to x} \frac{PR(y)}{\text{out}(y)}.$$

Power Iteration Algorithm for PageRank

We compute $\boldsymbol{p}(k) = (PR_k(1), \ldots, PR_k(N))^T \rightarrow \boldsymbol{r}$ iteratively via the **Power Method**:

Power Iteration Algorithm for PageRank

We compute $\boldsymbol{p}(k) = (PR_k(1), \ldots, PR_k(N))^T \to \boldsymbol{r}$ iteratively via the **Power Method**:

1. **Initialize ($k = 0$):** Set uniform initial probability distribution:

$$\boldsymbol{p}(0) = \frac{1}{N}\boldsymbol{e} \qquad \Longleftrightarrow \qquad PR_0(x) = \frac{1}{N} \quad (\forall x = 1, \ldots, N).$$

Power Iteration Algorithm for PageRank

We compute $\boldsymbol{p}(k) = (PR_k(1), \ldots, PR_k(N))^T \to \boldsymbol{r}$ iteratively via the **Power Method**:

1. **Initialize ($k = 0$):** Set uniform initial probability distribution:

$$\boldsymbol{p}(0) = \frac{1}{N}\boldsymbol{e} \qquad \Longleftrightarrow \qquad PR_0(x) = \frac{1}{N} \quad (\forall x = 1, \ldots, N).$$

2. **Iterate ($k = 0, 1, 2, \ldots$) until $\|\boldsymbol{p}(k+1) - \boldsymbol{p}(k)\|_1 < \varepsilon$:**

$$\forall x \in \{1, \ldots, N\}: \qquad PR_{k+1}(x) = \frac{1-\lambda}{N} + \lambda \sum_{y \to x} \frac{PR_k(y)}{\text{out}(y)}.$$

Example: Rapid Initial Convergence on the 11-Node Graph

Take $N = 11$, $\lambda = 0.85$ $(1 - \lambda = 0.15)$, and $PR_0(x) = \frac{1}{11} \approx 0.0909$:

Example: Rapid Initial Convergence on the 11-Node Graph

Take $N = 11$, $\lambda = 0.85$ $(1 - \lambda = 0.15)$, and $PR_0(x) = \frac{1}{11} \approx 0.0909$:

- **Step 1 for node** B (links from $C, D, E, F, G_1, G_2, G_3$):

$$PR_1(B) = \frac{0.15}{11} + 0.85 \cdot \frac{1}{11}\left(1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{2} + \frac{3}{2}\right) \approx \mathbf{0.31} \quad (\text{exact } r_B = 0.384).$$

Example: Rapid Initial Convergence on the 11-Node Graph

Take $N = 11$, $\lambda = 0.85$ $(1 - \lambda = 0.15)$, and $PR_0(x) = \frac{1}{11} \approx 0.0909$:

- **Step 1 for node** B (links from $C, D, E, F, G_1, G_2, G_3$):

$$PR_1(B) = \frac{0.15}{11} + 0.85 \cdot \frac{1}{11}\left(1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{2} + \frac{3}{2}\right) \approx \mathbf{0.31} \quad (\text{exact } r_B = 0.384).$$

- **Step 1 for node** C (only receives link from B with $\text{out}(B) = 1$):

$$PR_1(C) = \frac{0.15}{11} + 0.85\left(\frac{1}{11}\right) = \frac{1}{11} \approx 0.0909.$$

Example: Rapid Initial Convergence on the 11-Node Graph

Take $N = 11$, $\lambda = 0.85$ $(1 - \lambda = 0.15)$, and $PR_0(x) = \frac{1}{11} \approx 0.0909$:

- **Step 1 for node** B (links from $C, D, E, F, G_1, G_2, G_3$):

$$PR_1(B) = \frac{0.15}{11} + 0.85 \cdot \frac{1}{11}\left(1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{2} + \frac{3}{2}\right) \approx \mathbf{0.31} \quad (\text{exact } r_B = 0.384).$$

- **Step 1 for node** C (only receives link from B with $\text{out}(B) = 1$):

$$PR_1(C) = \frac{0.15}{11} + 0.85\left(\frac{1}{11}\right) = \frac{1}{11} \approx 0.0909.$$

- **Step 2 for node** C (B's high score propagates to C):

$$PR_2(C) = \frac{0.15}{11} + 0.85 \cdot PR_1(B) \approx \frac{0.15}{11} + 0.85 \times 0.31 \approx \mathbf{0.28} \quad (\text{exact } r_C = 0.343).$$

Problem with Sink (Dangling) Nodes

Consider $A \to B$ where B is a sink node ($\text{out}(B) = 0$), with $\lambda = 0.8$ and $PR_0(A) = PR_0(B) = 0.5$:

Problem with Sink (Dangling) Nodes

Consider $A \to B$ where B is a sink node ($\text{out}(B) = 0$), with $\lambda = 0.8$ and $PR_0(A) = PR_0(B) = 0.5$:

- Node A (no incoming links): $PR_1(A) = \dfrac{1-\lambda}{2} + \lambda \cdot 0 = \dfrac{0.2}{2} = 0.1.$

Problem with Sink (Dangling) Nodes

Consider $A \to B$ where B is a sink node ($\text{out}(B) = 0$), with $\lambda = 0.8$ and $PR_0(A) = PR_0(B) = 0.5$:

- Node A (no incoming links): $PR_1(A) = \dfrac{1-\lambda}{2} + \lambda \cdot 0 = \dfrac{0.2}{2} = 0.1.$
- Node B (incoming link from A, $\text{out}(A) = 1$):

$$PR_1(B) = 0.1 + 0.8 \times 0.5 = 0.5 \quad \Longrightarrow \quad PR_2(B) = 0.1 + 0.8 \times 0.1 = 0.18.$$

Problem with Sink (Dangling) Nodes

Consider $A \to B$ where B is a sink node ($\text{out}(B) = 0$), with $\lambda = 0.8$ and $PR_0(A) = PR_0(B) = 0.5$:

- Node A (no incoming links): $PR_1(A) = \dfrac{1-\lambda}{2} + \lambda \cdot 0 = \dfrac{0.2}{2} = 0.1.$
- Node B (incoming link from A, $\text{out}(A) = 1$):

$$PR_1(B) = 0.1 + 0.8 \times 0.5 = 0.5 \quad \Longrightarrow \quad PR_2(B) = 0.1 + 0.8 \times 0.1 = 0.18.$$

At steady state, $PR(A) + PR(B) = 0.1 + 0.18 = \mathbf{0.28} \neq \mathbf{1}$: probability mass leaks out through the zero column of sink node B!

Stochastic Fix for Sink Nodes

Solution: A walker at any sink node z ($\text{out}(z) = 0$, $z \to \emptyset$) teleports uniformly to all N nodes with probability $1/N$, replacing zero columns of $\boldsymbol{M}$ by $\frac{1}{N}\boldsymbol{e}$:

Stochastic Fix for Sink Nodes

Solution: A walker at any sink node z ($\text{out}(z) = 0$, $z \to \emptyset$) teleports uniformly to all N nodes with probability $1/N$, replacing zero columns of $\boldsymbol{M}$ by $\frac{1}{N}\boldsymbol{e}$:

$$\widetilde{\boldsymbol{M}} = \boldsymbol{M} + \frac{1}{N}\boldsymbol{e}\,\boldsymbol{d}^T, \qquad \text{where } d_z = \begin{cases} 1 & \text{if } \text{out}(z) = 0, \\ 0 & \text{otherwise.} \end{cases}$$

Stochastic Fix for Sink Nodes

Solution: A walker at any sink node z ($\text{out}(z) = 0$, $z \to \emptyset$) teleports uniformly to all N nodes with probability $1/N$, replacing zero columns of $\boldsymbol{M}$ by $\frac{1}{N}\boldsymbol{e}$:

$$\widetilde{\boldsymbol{M}} = \boldsymbol{M} + \frac{1}{N}\boldsymbol{e}\,\boldsymbol{d}^T, \qquad \text{where } d_z = \begin{cases} 1 & \text{if } \text{out}(z) = 0, \\ 0 & \text{otherwise.} \end{cases}$$

Matrix-free evaluation: Accumulate the sink probability mass into a single scalar S:

$$PR_{k+1}(x) = \frac{1-\lambda+\lambda S}{N} + \lambda \sum_{y \to x} \frac{PR_k(y)}{\text{out}(y)}, \qquad S = \boldsymbol{d}^T \boldsymbol{p}(k) = \sum_{z \to \emptyset} PR_k(z).$$

The Google Matrix G and Dominant Eigenvector

With the column-stochastic matrix $\widetilde{\boldsymbol{M}}$, the damped PageRank equation in vector form is:

$$\boldsymbol{p}(t+1) = \lambda \widetilde{\boldsymbol{M}} \boldsymbol{p}(t) + \frac{1-\lambda}{N} \boldsymbol{e} \qquad \xrightarrow{t \to \infty} \qquad \boldsymbol{r} = \lambda \widetilde{\boldsymbol{M}} \boldsymbol{r} + \frac{1-\lambda}{N} \boldsymbol{e}.$$

The Google Matrix G and Dominant Eigenvector

With the column-stochastic matrix $\widetilde{\boldsymbol{M}}$, the damped PageRank equation in vector form is:

$$\boldsymbol{p}(t+1) = \lambda \widetilde{\boldsymbol{M}} \boldsymbol{p}(t) + \frac{1-\lambda}{N}\boldsymbol{e} \qquad \xrightarrow{t\to\infty} \qquad \boldsymbol{r} = \lambda \widetilde{\boldsymbol{M}} \boldsymbol{r} + \frac{1-\lambda}{N}\boldsymbol{e}.$$

Since $\boldsymbol{e}^T \boldsymbol{r} = 1$, we can write $\frac{1-\lambda}{N}\boldsymbol{e} = \frac{1-\lambda}{N}\boldsymbol{e}(\boldsymbol{e}^T \boldsymbol{r})$ and define the **Google matrix** $\boldsymbol{G}$:

$$\boxed{\boldsymbol{G}\boldsymbol{r} = 1 \cdot \boldsymbol{r}, \qquad \boldsymbol{G} = \lambda \widetilde{\boldsymbol{M}} + \frac{1-\lambda}{N}\boldsymbol{e}\boldsymbol{e}^T \in \mathbb{R}^{N\times N}.}$$

Thus PageRank is the **dominant right eigenvector** of $\boldsymbol{G}$ for eigenvalue $\mu_1 = 1$.

Spectral Properties of the Google Matrix G

1. Strictly Positive Markov Matrix: $G_{ij} \geq \frac{1-\lambda}{N} > 0$ and $\boldsymbol{e}^T \boldsymbol{G} = \boldsymbol{e}^T$.

Spectral Properties of the Google Matrix $\boldsymbol{G}$

1. **Strictly Positive Markov Matrix:** $G_{ij} \geq \frac{1-\lambda}{N} > 0$ and $\boldsymbol{e}^T \boldsymbol{G} = \boldsymbol{e}^T$.
2. **Perron–Frobenius Theorem:** $\mu_1 = 1$ is a **strictly simple** dominant eigenvalue of $\boldsymbol{G}$, with a **unique** positive eigenvector $\boldsymbol{r} = (PR(1), \ldots, PR(N))^T > \boldsymbol{0}$.

Spectral Properties of the Google Matrix G

1. **Strictly Positive Markov Matrix:** $G_{ij} \geq \frac{1-\lambda}{N} > 0$ and $\boldsymbol{e}^T \boldsymbol{G} = \boldsymbol{e}^T$.
2. **Perron–Frobenius Theorem:** $\mu_1 = 1$ is a **strictly simple** dominant eigenvalue of $\boldsymbol{G}$, with a **unique** positive eigenvector $\boldsymbol{r} = (PR(1), \ldots, PR(N))^T > \mathbf{0}$.
3. **Spectral Gap & Fast Convergence:** The second eigenvalue satisfies $|\mu_2(\boldsymbol{G})| \leq \lambda < 1$, so Power Iteration converges geometrically at rate:

$$\|\boldsymbol{p}(k) - \boldsymbol{r}\|_1 \leq C\lambda^k \qquad \text{(for } \lambda = 0.85,\ 0.85^{50} \approx 3 \times 10^{-4}\text{, independent of } N!\text{)}.$$

Spectral Properties of the Google Matrix $\boldsymbol{G}$

1. **Strictly Positive Markov Matrix:** $G_{ij} \geq \frac{1-\lambda}{N} > 0$ and $\boldsymbol{e}^T \boldsymbol{G} = \boldsymbol{e}^T$.
2. **Perron–Frobenius Theorem:** $\mu_1 = 1$ is a **strictly simple** dominant eigenvalue of $\boldsymbol{G}$, with a **unique** positive eigenvector $\boldsymbol{r} = (PR(1), \ldots, PR(N))^T > \boldsymbol{0}$.
3. **Spectral Gap & Fast Convergence:** The second eigenvalue satisfies $|\mu_2(\boldsymbol{G})| \leq \lambda < 1$, so Power Iteration converges geometrically at rate:

   $$\|\boldsymbol{p}(k) - \boldsymbol{r}\|_1 \leq C\lambda^k \qquad \text{(for } \lambda = 0.85,\ 0.85^{50} \approx 3 \times 10^{-4}\text{, independent of } N!\text{)}.$$

4. **Sparse $O(|E| + N)$ Matrix–Vector Product:**

   $$\boldsymbol{p}(k+1) = \boldsymbol{G}\boldsymbol{p}(k) = \lambda \boldsymbol{M}\boldsymbol{p}(k) + \frac{1 - \lambda + \lambda(\boldsymbol{d}^T \boldsymbol{p}(k))}{N}\boldsymbol{e}.$$

Personalized PageRank (Recommender Systems)

Instead of jumping uniformly via $\frac{1}{N}\boldsymbol{e}$, teleportation can be biased toward a subset of **seed nodes** $\mathcal{S} \subset V$ (e.g., books, movies, or songs rated by a user, or topic-specific pages).

Personalized PageRank (Recommender Systems)

Instead of jumping uniformly via $\frac{1}{N}\boldsymbol{e}$, teleportation can be biased toward a subset of **seed nodes** $\mathcal{S} \subset V$ (e.g., books, movies, or songs rated by a user, or topic-specific pages).

We assign non-negative **preference weights** $\boldsymbol{c} \in \mathbb{R}^N$ supported on $\mathcal{S}$ ($c_i > 0$ for $i \in \mathcal{S}$, such as user ratings or $c_i = 1$, and $c_i = 0$ for $i \notin \mathcal{S}$), normalized to $\tilde{\boldsymbol{c}} = \frac{\boldsymbol{c}}{\boldsymbol{e}^T\boldsymbol{c}}$ ($\boldsymbol{e}^T\tilde{\boldsymbol{c}} = 1$). Replacing $\frac{1}{N}\boldsymbol{e}$ by $\tilde{\boldsymbol{c}}$ gives $\boldsymbol{r} = \lambda\widetilde{\boldsymbol{M}}\boldsymbol{r} + (1-\lambda)\tilde{\boldsymbol{c}}$, or:

$$\boxed{\widetilde{\boldsymbol{G}}\,\boldsymbol{r} = 1 \cdot \boldsymbol{r}, \qquad \widetilde{\boldsymbol{G}} = \lambda\widetilde{\boldsymbol{M}} + (1-\lambda)\tilde{\boldsymbol{c}}\,\boldsymbol{e}^T.}$$

Since $\boldsymbol{e}^T\widetilde{\boldsymbol{G}} = \boldsymbol{e}^T$, $\widetilde{\boldsymbol{G}}$ is column-stochastic and its dominant eigenvector $\boldsymbol{r}$ ranks all nodes **relative to the seed set $\mathcal{S}$ and weights $\boldsymbol{c}$**.